Uppsala University

Matematikprojekt i latex

Proof of The Pigeonhole Principle in Lean 4

Author:
Svante Sirviö

May 23, 2025



Abstract

This report constructs a lean 4 proof of a variation of the Pigeonhole principle. That is, any injective function f from $\{k \in \mathbb{N} | k < n\}$ to itself is also surjective. The proof relies only on the standard lean 4 library and demonstrates how a simple mathematical statement can be formalized using a proof assistant.

1 Introduction

1.1 Motivation

A proof assistant is a software tool that rigorously checks proofs of mathematical theorems for logical inconsistencies. Since it is infeasible to prove every known theorem from the axioms of mathematics by hand, a proof assistant relies on a computer to check that a given proof only relies on already proven theorems, which are in turn proven form the axioms. This means that in theory, as long as our axioms are consistent, a theorem proven in a proof assistant cannot have any mistakes.

However, this also makes it hard to actually prove theorems, as a proof has to be much more rigorous to be accepted by a proof assistant than by a human. To better understand how proof assistant proofs differ from normal human proofs, this report will prove a seemingly trivial statement in a proof assistant known as Lean 4.

1.2 Theorem

This report will prove a variation of the so called pigeonhole principle in the language Lean 4.

Definition. *Let* $[n]$ *denote the set* $\{k \in \mathbb{N} \mid k < n\}$

Remark. *This report uses the convention that* $0 \in \mathbb{N}$

Theorem (Pigeonhole-principle)**.** *Let* $n \in \mathbb{N}$ *and let* $f : [n] \to [n]$ *be an injective function. Then* f *is surjective.*

1.3 Background

Lean 4 is a functional programming language that is used as a proof assistant for mathematicians. For a detailed explanation of how to code in lean 4, the reader is referred to *Functional Programming in Lean* [2]. This section gives a short explanation of the most important aspects, as well as a description of the so called tactic mode.

One of the main differences between Lean 4 and programming languages like python is that functions have no side-effects. That is, functions act like mathematical functions in that they only return a single value and can't perform other actions, like printing to the console. This means that programs are completely deterministic which is taken advantage of to make a proof assistant. This can be avoided with monads, but they are not relevant to this report and therefore left out.

In Lean, everything has a type. For example `1` is of type `Nat` and a real function is of type `Real → Real`. The theorem below has type `(a : Nat) : a + 0 = a`. To prove a theorem one has to construct an instance of that type. Normally

this is done by composing other types, but to make the proofs more human-readable, Lean has a so called tactic mode.

A proof in Leans tactic mode can look like the following:

```
theorem mul_zero (a : Nat) : a + 0 =
    a :=
  by
  rfl
```

```
1 goal
a : Nat
⊢ a + 0 = a
```

To the left is the theorem and the proof and to the right is the goal state. The goal state shows variables and assumptions one can use, as well as the goal that has to be proven. Inside the proof, tactics are used to prove the theorem. Here is a list of some tactics and what they do.

`rfl`	Proves an equality if both sides are equal by definition
`symm`	Flips an equality
`rw`	If `a = b`, then `rw` replaces `a` with `b`
`intro`	If a goal is of the form `P` $\rightarrow$ `Q`, then it introduces a as a hypothesis `P` and changes the goal to `Q`
`cases`	If a variable is defined as one of many constructors, then it breaks down into different cases for each of the constructors
`induction`	Simmilar to `cases`, but introduces an induction hypothesis for certain cases
`by_cases`	Different from `cases`. Makes two cases based on if any arbitrary hypothesis is true or false
`contradiction`	Used for proving contradictions
`apply`	If the goal is `Q` and one theorem or hypothesis is `P` $\rightarrow$ `Q` then it changes the goal to `P`

Some other keywords that are commonly used inside leans tactic mode include the following.

`let`	Defines a new variable
`have`	Defines a new hypothesis, if it can be proven
`exists`	Proves that there exists an element that satisfies a certain property

Finally, one definition that will be used heavily in the proof is the definition of $[n]$. In lean this set is defined as the type `Fin n`.

```
structure Fin (n : Nat) where
val : Nat
isLt : val < n
```

Thus, an element of `Fin n` is a natural number `val` and a proof that `val < n`. A consequence of this definition is that, for example `1 : Fin 2` and `1 : Fin 3` are not equal by definition, since the first contains a proof that `1 < 2` whereas the ladder contains a proof that `1 < 3`. This will be important later in the proof.

2 Proof

As a reminder, the following is the theorem that will be proven.

Theorem (Pigieonhole-Principle)**.** *Let $n \in \mathbb{N}$ and let $f : [n] \to [n]$ be an injective function. Then f is surjective.*

Written in lean 4 the statement looks like the following.

```
theorem surj_of_inj_fin (f : Fin n →
    Fin n) (h_inj : Injective f) :
    Surjective f
```

```
1 goal
n : Nat
f : Fin n → Fin n
h_inj : Injective f
⊢ Surjective f
```

The theorem will be proven with an induction argument. The case $n = 0$ is a special case, but the the theorem still holds. Therefore the induction argument proves two base cases, $n = 0$ and $n = 1$ and then assumes $n \geq 2$ for the induction step.

```
by
induction n with
|zero => -- Proof for n=0
|succ n' ih =>
cases n' with
|zero => -- Proof for n=1
|succ a => -- Proof for n>1
```

```
3 goals
case zero
n : Nat
f : Fin 0 → Fin 0
h_inj : Injective f
⊢ Surjective f

case succ.zero
n : Nat
ih : ∀ (f : Fin 0 →
    Fin 0), Injective
    f → Surjective f
f : Fin (0 + 1) →
    Fin (0 + 1)
h_inj : Injective f
⊢ Surjective f

case succ.succ
n a : Nat
ih : ∀ (f : Fin (a +
    1) → Fin (a +
    1)), Injective f
    → Surjective f
f : Fin (a + 1 + 1)
    → Fin (a + 1 + 1)
h_inj : Injective f
⊢ Surjective f
```

As shown in the goal state, there are 3 goals to prove. The case `n = zero`, the case `n = succ (zero)` and the case `n = succ (succ a)` where `a` is any natural number. These cases are the same as the cases described above.

2.1 Case: $n = 0$

For the case $n = 0$, the lean state is the following.

```
|zero => --Proof for n = 0
```

```
1 goal
n : Nat
f : Fin n → Fin n
h_inj : Injective f
⊢ Surjective f
```

This is a special case, but the theorem holds since functions from the empty set to itself are bijective by default. First we prove that [0] is empty.

```
--Fin 0 is empty
have h_empty_fin_0 : ¬Nonempty (Fin
    0) :=
      by
      intro h
      have a : Fin 0 :=
    Classical.choice h
      have h : a.val < 0 := a.isLt
      contradiction
```

```
1 goal
n : Nat
f : Fin 0 → Fin 0
h_inj : Injective f
⊢ ¬Nonempty (Fin 0)
```

This is proven by assuming an element exists and noting that this element has to be a natural number smaller than zero, which is of course a contradiction

Now the case is proven by noting that any function to an empty type is surjective by the lemma `surj_to_empty`. The proof of this lemma can be seen in the source code. [3]

```
exact surj_to_empty f h_empty_fin_0
```

```
No goals
```

Altogether it looks like this

```
|zero =>
    --Fin 0 is empty
    have h_empty_fin_0 : ¬Nonempty
    (Fin 0) :=
        by
        intro h
        have a : Fin 0 :=
    Classical.choice h
        have h : a.val < 0 := a.isLt
        contradiction

    --Function to empty type is
    surjective
    exact surj_to_empty f
    h_empty_fin_0
```

```
No goals
```

This proves the case $n = 0$.

2.2 Case: $n = 1$

For the case $n = 1$ the lean state is the following.

```
by
induction n with
|zero => -- Proven above
|succ n' ih =>
cases n' with
|zero => -- Proof for n=1
```

```
1 goal
case succ.zero
n : Nat
ih : ∀ (f : Fin 0 →
    Fin 0), Injective
    f → Surjective f
f : Fin (0 + 1) →
    Fin (0 + 1)
h_inj : Injective f
⊢ Surjective f
```

To prove this case we first prove that the only element in [1] is 0.

```
--If a : Fin 1 then a = 0
have h_zero_only (a : Fin 1) : a = ⟨
    0, Nat.zero_lt_one⟩ :=
    by
    have h_a_lt_one : a.val < 1 :=
    a.isLt
    have h_a_ge_zero : 0 ≤ a.val :=
    Nat.zero_le a.val
    have h_a_le_zero : a.val ≤ 0 :=
    Nat.le_of_lt_succ h_a_lt_one
    have h_a_val_eq : a.val = 0 :=
    Nat.le_antisymm h_a_le_zero
    h_a_ge_zero
    apply Fin.eq_of_val_eq
    rw[Fin.val_mk Nat.zero_lt_one]
    exact h_a_val_eq
```

```
1 goal
n : Nat
ih : ∀ (f : Fin 0 →
    Fin 0), Injective
    f → Surjective f
f : Fin (0 + 1) →
    Fin (0 + 1)
h_inj : Injective f
a : Fin 1
⊢ a = ⟨0,
    Nat.zero_lt_one⟩
```

This is done by first showing $a < 1$ and $a \geq 0$. Thus $a = 0$.

Now we can prove that f is surjective

```
intro a
have h_a_eq_zero : a = ⟨0,
    Nat.zero_lt_one⟩ := h_zero_only a

exists ⟨0, Nat.zero_lt_one⟩

have h_f_eq_zero : (f ⟨0,
    Nat.zero_lt_one⟩) = ⟨0,
    Nat.zero_lt_one⟩ := h_zero_only
    (f ⟨0, Nat.zero_lt_one⟩)

rw[h_a_eq_zero, h_f_eq_zero]
```

```
1 goal
case succ.zero
n : Nat
ih : ∀ (f : Fin 0 →
    Fin 0), Injective
    f → Surjective f
f : Fin (0 + 1) →
    Fin (0 + 1)
h_inj : Injective f
h_zero_only : ∀ (a :
    Fin 1), a = ⟨0,
    Nat.zero_lt_one⟩
⊢ Surjective f
```

This is done by assuming $a \in [1]$. Then $a = 0$ by the lemma above. Also $f(0) = 0$ since this is the only option. So we find $f(0) = a$. Thus f is surjective. This proves the case $n = 1$.

2.3 Case: $n \geq 2$

For the induction step the following shows the lean state.

```
by
induction n with
|zero => -- Proof for n=0
|succ n' ih =>
cases n' with
|zero => -- Proof for n=1
|succ a => -- Proof for n>1
```

```
1 goal
case succ.succ
n a : Nat
ih : ∀ (f : Fin (a +
    1) → Fin (a +
    1)), Injective f
    → Surjective f
f : Fin (a + 1 + 1)
    → Fin (a + 1 + 1)
h_inj : Injective f
⊢ Surjective f
```

To prove the theorem, first the following function is constructed

$$g : [n-1] \to [n-1], \quad i \mapsto \begin{cases} f(i) & \text{if} \quad f(i) < n \\ f(n) & \text{if} \quad f(i) = n \end{cases} \tag{1}$$

In lean 4 this function is defined as below

```
def downCastFunc (h_pos : n > 0) (f : Fin (n+1) → Fin (n+1)) :
    Fin n → Fin n :=
  fun i =>
    have h_succ_n_pos : n+1 > 0 := Nat.zero_lt_succ n
    let i_up : Fin (n+1) := @cast (n) (n+1) h_succ_n_pos i
        if _ : (f i_up) < (n) then
          @cast (n+1) (n) h_pos (f i_up)
        else
          @cast (n+1) (n) h_pos (f ⟨(n), Nat.lt_succ_self n⟩)
```

Since lean treats, for example `1 : Fin(3)` and `1 : Fin(4)` as different elements, the definition makes use of the `cast` function, which converts a `Fin(n)` to a `Fin(m)`.

When f is an injective function g will also be injective. This lemma is used below and called `dcf_inj_of_inj`. The proof of this lemma can be seen in the source code. [3]

```
|succ a =>
      have h_succ_a_pos : 0 < (a+1) :=
     Nat.zero_lt_succ a

      let g : Fin (a+1) → Fin (a+1) :
    = downCastFunc (a+1) h_succ_a_pos
    f

      --g is injective as proven above
      have h_g_inj : Injective g :=
    dcf_inj_of_inj (a+1) h_succ_a_pos
    f h_inj

      --g is surjective by induction
    hypothesis
      have h_g_surj : Surjective g :=
    ih g h_g_inj
```

```
1 goal
case succ.succ
n a : Nat
ih : ∀ (f : Fin (a +
    1) → Fin (a +
    1)), Injective f
    → Surjective f
f : Fin (a + 1 + 1)
    → Fin (a + 1 + 1)
h_inj : Injective f
h_succ_a_pos : 0 < a
    + 1
g : Fin (a + 1) →
    Fin (a + 1) :=
    downCastFunc (a +
    1) h_succ_a_pos f
h_g_inj : Injective g
h_g_surj :
    Surjective g
⊢ Surjective f
```

The code above constructs g and proves that it is injective from the lemma and thus surjective by the induction hypothesis.

Now the proof that f is surjective is divided into two cases. For $b = n - 1$ and $b < n - 1$. Both of these cases are proven with separate lemmas. `surj_of_eq_of_dcf_surj` and `surj_of_lt_of_dcf_surj`. Again the proofs of theese lemmas are found in the source code. [3]

```
--Prove that f is surjective
intro b

let a_succ_fin : Fin (a+2) := ⟨(a+1),
    Nat.lt_succ_self (a+1)⟩

by_cases h_b_eq : b = a_succ_fin·
 have h_b_vals_eq : ↑b = (a+1) :=
    Fin.val_eq_of_eq h_b_eq
    exact surj_of_eq_of_dcf_surj
    (a+1) h_succ_a_pos b h_b_vals_eq
    f h_inj h_g_surj·

 have h_b_lt : b.val < (a+1) :=
    val_lt_of_val_not_eq b
    (val_ne_of_ne b a_succ_fin h_b_eq)
    exact surj_of_lt_of_dcf_surj
    (a+1) h_succ_a_pos b h_b_lt f
    h_inj h_g_surj
```

```
1 goal
case succ.succ
n a : Nat
ih : ∀ (f : Fin (a +
    1) → Fin (a +
    1)), Injective f
    → Surjective f
f : Fin (a + 1 + 1)
    → Fin (a + 1 + 1)
h_inj : Injective f
h_succ_a_pos : 0 < a
    + 1
g : Fin (a + 1) →
    Fin (a + 1) :=
    downCastFunc (a +
    1) h_succ_a_pos f
h_g_inj : Injective g
h_g_surj :
    Surjective g
b : Fin (a + 1 + 1)
⊢ ∃ a_1, f a_1 = b
```

This proves the induction step and thus the whole theorem is proven. The full proof found uninterrupted in the source code. [3]

Acknowledgements

I learned to code in lean 4 and use lean as a proof assistant by studying several sources. The book *Functional Programming in Lean* [2] and the games on *Lean Game Server* [1] were invaluable for my understanding of the language.

References

[1] Alexander Bentkamp and Jon Eugster. *Lean Game Server*. Accesed multiple times in 2025. 2025. URL: https://adam.math.hhu.de/#/.

[2] David Thrane Christiansen. *Functional Programming in Lean*. Accesed between January and Mars 2025. 2024. URL: https://lean-lang.org/functional_programming_in_lean/.

[3] Svante Sirviö. *My Math*. Commit: 9e9c494b3492ab7e8f97306b4556b09318cd2767. May 2025. URL: https://github.com/Svirre03/MyMath.